

AI for Bharat Hackathon

Powered by **aws**



Team Name : Cyber Scorpion

Team Leader : Bhavik Patel

Team Member : Hem Bhavsar

Problem : AI-Powered Clinical Document Summarisation

Brief about the Idea:

MediAssist AI – Clinical Document Summarization

Healthcare professionals in India spend significant time translating complex clinical documents into language patients can understand. MediAssist AI solves this using:

Ollama LLM (llama3.2:3b) running privately on AWS EC2 – no data leaves your infrastructure

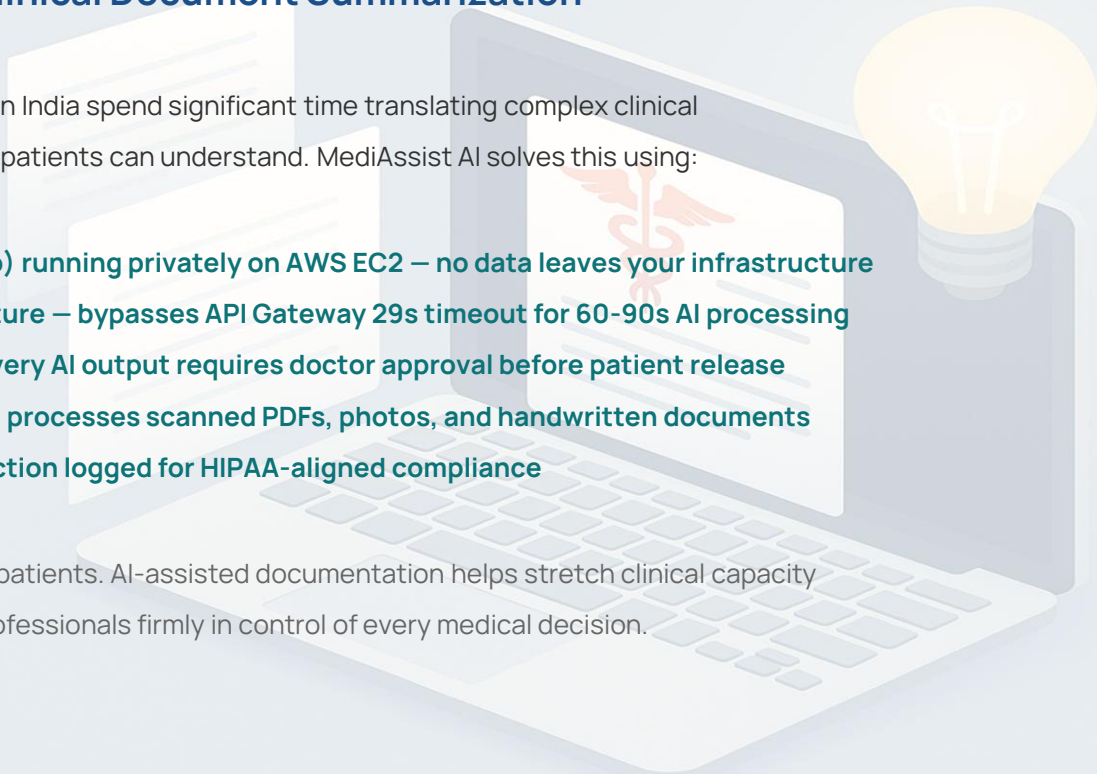
Async Lambda architecture – bypasses API Gateway 29s timeout for 60-90s AI processing

Human-in-the-loop – every AI output requires doctor approval before patient release

Amazon Textract OCR – processes scanned PDFs, photos, and handwritten documents

Full audit trail – every action logged for HIPAA-aligned compliance

India has 1 doctor per 834 patients. AI-assisted documentation helps stretch clinical capacity while keeping qualified professionals firmly in control of every medical decision.



AI + AWS Value Proposition

Why AI is Required

Manual summarization of clinical records takes 15-20 mins per patient. With 700M+ outpatient visits/year in India, AI is the only scalable solution. Rule-based systems cannot handle the variability of clinical language — LLMs excel at this specific task.

How AWS Services Power the Architecture

Lambda (x5 functions) | API Gateway HTTP API | DynamoDB (3 tables)
S3 (storage + hosting) | Amazon Textract (OCR) | CloudWatch (monitoring)
EC2 t3.large running Ollama LLM — private inference, no third-party API costs

Value the AI Layer Adds

80% reduction in time to create patient-friendly explanations
Confidence scoring (High/Medium/Low) guides doctor review priority
Consistent output quality regardless of clinician workload or fatigue



Key Features

Document Processing

PDF (text + scanned), DOCX, TXT
JPG, PNG, TIFF, BMP, WebP images
Amazon Textract OCR fallback
Up to 10MB file size

AI Generation

Professional clinical summary
Patient-friendly explanation
Confidence score (High/Med/Low)
AI safety disclaimer auto-added

Compliance & Audit

Full timestamped audit trail
Every edit tracked by reviewer
Export as TXT or JSON



Async Architecture

No API Gateway timeout (29s limit)
Worker Lambda: 600s timeout
React polls /status every 5s
Instant doc_id response to user

Human Oversight

Doctor reviews every AI output
Inline editing before approval
No patient release without sign-off
Approval logged with reviewer ID

Monitoring

CloudWatch live dashboard
Error alerts on all 5 Lambdas
Ollama duration tracking

Process Flow Diagram

4-Step Workflow

STEP 1 – UPLOAD

Clinician uploads PDF / DOCX / Image | Consent checkbox required | File validated (type + size)
Lambda extracts text: PyPDF2 (PDF) / python-docx (DOCX) / Textract OCR (images + scanned)

STEP 2 – ASYNC PROCESSING

Process Lambda saves to S3 + DynamoDB with status=PROCESSING | Returns doc_id in <1s
Worker Lambda invoked async | Calls Ollama (llama3.2:3b) | Updates DynamoDB to PENDING

STEP 3 – HUMAN REVIEW

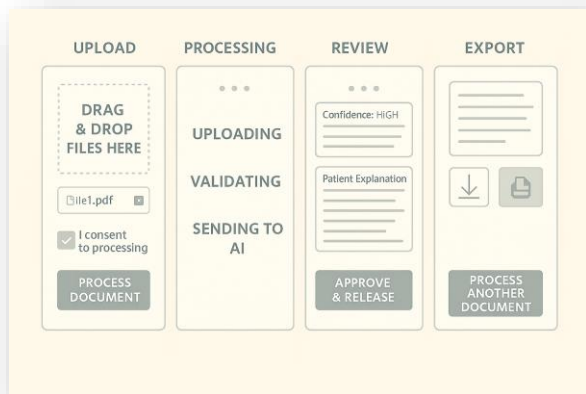
React polls GET /status every 5s | Shows spinner with elapsed time during AI processing
Doctor edits professional summary + patient explanation | All edits logged in audit trail

STEP 4 – APPROVE + EXPORT

POST /approve with edited content | Status becomes APPROVED | Patient explanation released
Export as TXT summary or JSON | Print audit log | Full compliance trail saved



UI Wireframes – 4 Panel Workflow



Panel 1: Upload

Drag-and-drop zone

File type badges: PDF / TXT / DOCX / JPG / PNG

File preview with size + remove button

Consent checkbox (required)

Process Document button

Panel 2: Processing

Upload progress animation (3 steps)

Uploading / Validating / Sending to AI

Transitions to Review panel in ~1s

Panel 3: Review

AI spinner while Ollama processes (30-90s)

Confidence score bar (High/Med/Low)

Editable Professional Summary textarea

Editable Patient Explanation textarea

Approve + Release button

Panel 4: Export

Approved patient explanation card

Download TXT / Copy JSON / Print

Full audit trail table

Process Another Document button

Architecture Diagram

AWS Serverless Architecture – ap-south-1 (Mumbai)

FRONTEND (React + Vite) --> API GATEWAY (HTTP API v2) --> LAMBDA LAYER

Lambda Functions:

MediAssist-Process (30s) Receives upload, extracts text, saves to S3+DynamoDB, fires Worker async

MediAssist-Worker (600s) Calls Ollama LLM on EC2, updates DynamoDB – no API GW timeout!

MediAssist-Status (30s) Polled by React every 5s – returns PROCESSING / PENDING / FAILED

MediAssist-Approve (30s) Doctor approval workflow, saves edited summaries, logs audit event

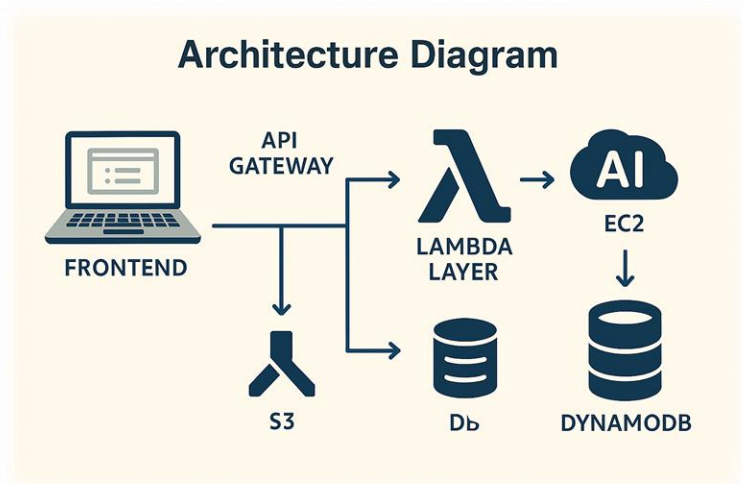
MediAssist-Audit (30s) Returns full compliance audit log for a document

Storage:

S3: mediassistai-documents (uploads) | mediassistai-frontend (React static hosting)

DynamoDB: MediAssist-Results | MediAssist-Users | MediAssist-AuditLog (PAY_PER_REQUEST)

EC2 t3.large: Ollama + llama3.2:3b | Textract: OCR for images + scanned PDFs



Technologies Utilized

Frontend

React 19.2 + Vite

TailwindCSS

React Context API (state)

Fetch API + polling pattern

Backend (Lambda – Python 3.11)

boto3 (S3, DynamoDB, Lambda, Textract)

PyPDF2 (PDF text extraction)

python-docx (DOCX extraction)

Amazon Textract (OCR)

AI / LLM

Ollama (self-hosted on EC2)

llama3.2:3b model

600s Lambda timeout

AWS Services

Lambda (x5 functions)

API Gateway HTTP API v2

DynamoDB (3 tables)

S3 (storage + hosting)

EC2 t3.large (Ollama)

Amazon Textract (OCR)

CloudWatch (monitoring)

IAM (least-privilege roles)

Deployment

deploy.sh (full automation)

Region: ap-south-1 Mumbai

Vite build to S3 static site



Estimated Implementation Cost

AWS Cost Estimate – Hackathon Demo Scale

Service	Monthly Cost (est.)	Notes
EC2 t3.large (Ollama)	~\$60/month	On-demand, stop when not demoing
Lambda (5 functions)	~\$0-2/month	Free tier: 1M requests, 400K GB-s
API Gateway HTTP API	~\$0-1/month	\$1.00 per million API calls
DynamoDB (PAY_PER_REQUEST)	~\$0-1/month	Free tier covers demo volume
S3 (documents + frontend)	~\$0.05/month	Minimal storage for prototype
Amazon Textract	~\$0-2/month	\$1.50 per 1,000 pages (OCR)
CloudWatch	~\$0/month	Free tier covers log storage

Total (Prototype) **~\$63/month** **Dominated by EC2 Ollama instance**

-Production scaling: Switch EC2 to g4dn.xlarge (GPU) for 10x faster inference at ~\$200/month

-or replace Ollama with Amazon Bedrock (pay-per-token, no EC2 management).

Prototype Snapshots

Live Prototype – Deployed on AWS (ap-south-1)

Screen 1: Upload Panel

Drag-and-drop zone accepts PDF / DOCX / TXT / JPG / PNG | File preview + consent checkbox

Screen 2: Processing Panel

3-step animation (Upload / Validate / Send to AI) | Transitions in under 1 second

Screen 3: Review Panel (AI Result)

Confidence score bar | Editable professional summary | Editable patient explanation

AI polls every 5s – shows elapsed time | Approve & Release button (disabled until reviewed)

Screen 4: Export Panel

Approved patient explanation | Download TXT / Copy JSON / Print Audit Log

Full audit trail: UPLOAD > GENERATE > EDIT > APPROVE – all timestamped

Frontend URL: <http://mediassistai-frontend.s3-website.ap-south-1.amazonaws.com>

Performance Report & Benchmarking

Optimization Path

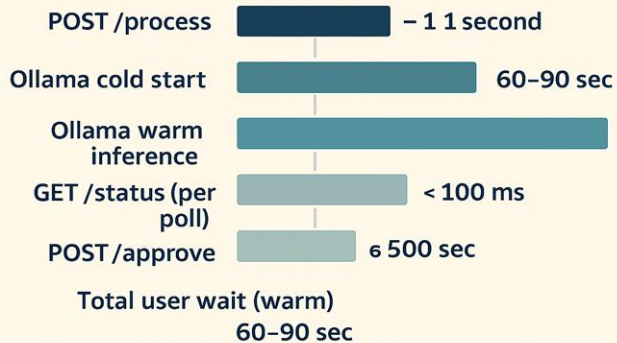
Pre-warm Ollama before demo: `curl EC2:11434/api/generate` (loads model into RAM)

Reduce input to 3000 chars (vs 8000): cuts inference time to ~40-50 seconds

GPU EC2 (g4dn.xlarge): ~10x faster inference | Amazon Bedrock: fully managed

CloudWatch dashboard: live monitoring of all 5 Lambda functions + DynamoDB

End-to-End Latency Breakdown



Future Development Roadmap

Short Term (3-6 months)

- Multilingual support
- Hindi, Tamil, Gujarati, Bengali
- AWS Cognito authentication
- Role-based access (Doctor/Nurse/Admin)
- GPU EC2 for < 10s inference
- g4dn.xlarge with llama3.2:7b

Medium Term (6-12 months)

- HL7 FHIR integration
- Direct EHR system connectivity
- WhatsApp patient delivery
- React Native mobile app

Long Term (12+ months)

- Amazon Bedrock migration
- Claude 3 Haiku for lower cost + speed
- Voice-to-document support
- Ayushman Bharat integration
- National Health Stack alignment

Responsible AI Commitments

- Bias audit across demographics
- Clinical validation studies
- CDSCO regulatory alignment
- HIPAA + India DPDP Act compliance
- Privacy impact assessments

Prototype Assets

Submission Assets – Team Cyber Scorpion

GitHub Repository (Public)

github.com/BhavikPatel/AI-for-Healthcare-Life-Sciences

Includes: All Lambda functions | React frontend | deploy.sh | README | SECURITY.md

Live Demo URL

<http://mediassistai-frontend.s3-website.ap-south-1.amazonaws.com>

API: <https://68nnaciiv6.execute-api.ap-south-1.amazonaws.com/prod>

Demo Video (under 3 minutes)



Shows: Upload PDF > AI Processing > Review & Approve > Export + Audit Trail

Key Files:

backend/lambda/ (5 Python files) | frontend/src/panels/ (4 React panels) | infra/deploy.sh

Innovation partner **I12S**
HACKZESTLE

Media partner **YOURSTORY**

AI for Bharat Hackathon

Powered by **aws**

Thank You

